

SlabHash

A source-code-driven guide to the GPU implementation

Repository audited: owensgroup/SlabHash, branch master, commit eec7135 (master checkout audited 2026-09-06)

Evidence policy. This is a reverse engineering of the checked-out source and its pinned SlabAlloc submodule. It avoids generic textbook material. Every excerpt below is taken from the repository and carries a path, symbol, and purpose.

Scope. The global header says the pair representation only works with up to 32-bit key/values. ConcurrentMap is the complete primary implementation. ConcurrentSet is partial; PhaseConcurrentMap has a type declaration but no specialization in this checkout. Dynamic means growing overflow chains, not resizing the fixed base bucket array.

Contents

- 1. Big picture
- 2. Repository structure
- 3. Architecture and ownership
- 4. Core data structures
- 5. Slab allocator
- 6. ConcurrentMap operations
- 7. Warp execution and concurrency
- 8. GpuSlabHashContext
- 9. Variants
- 10. CUDA search kernel
- 11. Memory layout
- 12. Benchmarks, tests and build
- 13. Complete call/data flow
- 14. Reading order
- SlabHash - Complete Mental Model

1. Big picture

What SlabHash is. README.md calls it a warp-oriented dynamic hash table for GPUs. The source implements that using an owning GpuSlabHash, a device-copyable GpuSlabHashContext, fixed 128-byte base slabs, allocator-owned overflow slabs, and warp-cooperative operations.

Why not a CPU-style table here. A one-thread chain walk wastes sibling GPU lanes and yields scattered accesses. SlabHash makes all 32 lanes load the 32 words of one slab, then uses ballots/shuffles to identify a match, free slot, or next link. This is concrete in searchKey, insertPair and warpAllocate.

Mental model. A bucket is the first slab itself. A key hashes to its base slab. A warp scans its 15 key/value pairs. If full, lane 31 links an allocator-provided 32-word overflow slab. The base bucket count does not grow; collision chains do.

src/slab_hash_global.cuh

Function/class: concurrent_slab

What it is doing: Defines the ConcurrentMap slab layout used for base and overflow slabs.

```
struct __align__(32) concurrent_slab {  
    static constexpr uint32_t NUM_ELEMENTS_PER_SLAB = 15u;  
    key_value_pair<KeyT, ValueT> data[NUM_ELEMENTS_PER_SLAB];  
    uint32_t ptr_index[2];  
};
```

For 32-bit KeyT/ValueT, 15 pairs consume words 0..29; even words are keys and following odd words are values. Map operations use word 31 as the next-slab link. ptr_index[0] at word 30 exists in the struct but is not used by the ConcurrentMap routines inspected.

CUDA word/lane	Slab interpretation
0,2,...,28	data[0].key ... data[14].key
1,3,...,29	data[0].value ... data[14].value
30	ptr_index[0]
31	ptr_index[1] - encoded next overflow address

2. Repository structure

Path	Actual role
src/	Header-only templates: global types, map/set specializations, warp operations, kernels and host wrappers; slab_hash.cuh is the umbrella.
SlabAlloc/	Pinned allocator submodule: GPU pool owner, device context, bitmaps and address decoding.
bench/	Benchmark executable, experiment scenarios, workload generator and Python driver.
test/	ConcurrentMap GoogleTests and standalone map/set/iterator demos.
ThirdParty/	rapidjson for benchmark JSON; googletest for cmap_test.
CMakeLists.txt	FindCUDA setup, gencode flags, includes and targets.

slab_hash_global.cuh defines sentinels, layouts, variants and allocator aliases. slab_hash.cuh includes class declarations, per-warp implementation, device kernels and host method bodies. gpu_hash_table.cuh is a convenience owner used by tests/benchmarks, not the core hash implementation.

src/slab_hash.cuh

Function/class: umbrella include sequence

What it is doing: Shows the dependency assembly sequence for templates and device functions.

```
// class declaration:
#include "concurrent_map/cmap_class.cuh"
#include "concurrent_set/cset_class.cuh"
#include "slab_iterator.cuh"

// warp implementations of member functions:
#include "concurrent_map/warp/delete.cuh"
#include "concurrent_map/warp/insert.cuh"
#include "concurrent_map/warp/search.cuh"
#include "concurrent_map/warp/count.cuh"

#include "concurrent_set/cset_warp_operations.cuh"

// helper kernels:
#include "concurrent_map/device/build.cuh"
#include "concurrent_map/device/concurrent_kernel.cuh"
#include "concurrent_map/device/delete_kernel.cuh"
#include "concurrent_map/device/misc_kernels.cuh"
#include "concurrent_map/device/search_kernel.cuh"
#include "concurrent_map/device/count_kernel.cuh"
#include "concurrent_set/cset_helper_kernels.cuh"

// implementations:
```

3. Architecture and ownership

GpuSlabHash owns `d_table_`, the base-bucket allocation. GpuSlabHashContext owns no GPU allocation; kernels receive it by value. SlabAllocLight owns `d_super_blocks_`, its overflow slab pool. `gpu_hash_table` owns the host C++ map/allocator pointers and `d_key_/d_value_/d_query_/d_result_/d_count_` staging arrays.

Object	Owns / references	Used where
<code>gpu_hash_table</code>	Allocator/map C++ objects plus CUDA staging arrays.	Host tests/benchmarks.
<code>GpuSlabHash</code>	<code>d_table_</code> ; non-owning allocator pointer; context value.	Host constructor/launch API.
Context	Hash metadata, <code>d_table_</code> reference, copied allocator context.	CUDA kernel/device API.
<code>SlabAllocLight</code>	<code>d_super_blocks_</code> allocation and allocator context.	Host construction/device allocation.

src/concurrent_map/cmap_class.cuh

Function/class: GpuSlabHash constructor

What it is doing: Allocates all base buckets, fills fields with empty sentinels, and initializes the context.

```
slab_unit_size_ =
    GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::getSlabUnitSize();

// allocating initial buckets:
CHECK_CUDA_ERROR(cudaMalloc((void**)&d_table_, slab_unit_size_ * num_buckets));

CHECK_CUDA_ERROR(cudaMemset(d_table_, 0xFF, slab_unit_size_ * num_buckets));

// creating a random number generator:
if (!identity_hash) {
    std::mt19937 rng(seed ? seed : time(0));
    hf_.x = rng() % PRIME_DIVISOR_;
    if (hf_.x < 1)
        hf_.x = 1;
    hf_.y = rng() % PRIME_DIVISOR_;
} else {
    hf_ = {0u, 0u};
}

// initializing the gpu_context:
gpu_context_.initParameters(
    num_buckets_, hf_.x, hf_.y, d_table_, dynamic_allocator_->getContextPtr());
}

~GpuSlabHash() {
```

The all-ones memset establishes `EMPTY_KEY`, `EMPTY_VALUE`, `EMPTY_PAIR_64` and `EMPTY_INDEX_POINTER` at once. Growth does not realloc `d_table_`; it gets linked units from `SlabAlloc`. The map destructor frees `d_table_`, and `SlabAllocLight`'s destructor frees its independent pool.

src/gpu_hash_table.cuh

Function/class: `gpu_hash_table` constructor

What it is doing: Creates the allocator then passes its non-owning pointer into `GpuSlabHash`.

```
dynamic_allocator_ = new DynamicAllocatorT();

// slab hash:
slab_hash_ = new GpuSlabHash<KeyT, ValueT, SlabHashT>(
    num_buckets_, dynamic_allocator_, device_idx_, seed_, identity_hash_);
if (verbose) {
    std::cout << slab_hash_->to_string() << std::endl;
}
```

4. Core data structures

src/slab_hash_global.cuh

Function/class: ConcurrentMapT

What it is doing: Supplies map sentinels, 32-word bucket stride, key-lane mask and slab type.

```
class ConcurrentMapT {
public:
    // fixed parameters for the data structure
    static constexpr uint32_t A_INDEX_POINTER = 0xFFFFFFFF;
    static constexpr uint32_t EMPTY_INDEX_POINTER = 0xFFFFFFFF;
    static constexpr uint32_t BASE_UNIT_SIZE = 32;
    static constexpr uint32_t REGULAR_NODE_ADDRESS_MASK = 0x30000000;
    static constexpr uint32_t REGULAR_NODE_DATA_MASK = 0x3FFFFFFF;
    static constexpr uint32_t REGULAR_NODE_KEY_MASK = 0x15555555;

    using SlabTypeT = concurrent_slab<KeyT, ValueT>;

    static std::string getTypeName() { return std::string("ConcurrentMap"); }
};

template <typename KeyT>
class ConcurrentSetT {
```

A_INDEX_POINTER is a register-only instruction to use a base bucket. EMPTY_INDEX_POINTER is the stored no-link value. BASE_UNIT_SIZE=32 gives the d_table_stride. REGULAR_NODE_KEY_MASK=0x15555555 selects even key lanes and excludes values plus link words.

Base and overflow pointer access

src/concurrent_map/cmap_class.cuh

Function/class: getPointerFromSlab / getPointerFromBucket

What it is doing: Maps a base bucket or encoded overflow address plus lane to a GPU word pointer.

```
uint32_t* getPointerFromSlab(
    const SlabAddressT& slab_address,
    const uint32_t laneId) {
    return global_allocator_ctx_.getPointerFromSlab(slab_address, laneId);
}

__device__ __forceinline__ uint32_t* getPointerFromBucket(const uint32_t bucket_id,
    const uint32_t laneId) {
    return reinterpret_cast<uint32_t*>(d_table_) +
        bucket_id * ConcurrentMapT<KeyT, ValueT>::BASE_UNIT_SIZE + laneId;
}

private:
    // this function should be operated in a warp-wide fashion
    // TODO: add required asserts to make sure this is true in tests/debugs
    __device__ __forceinline__ SlabAllocAddressT allocateSlab(const uint32_t& laneId) {
```

Base access is direct d_table_ + bucket*32 + lane. Overflow access decodes a SlabAllocAddressT then adds lane. This is why the same warp scan code works for base and collision slabs.

5. Slab allocator

DynamicAllocatorT is SlabAllocLight<8,32,1>: 256 memory blocks per super block, 32 super blocks, and a 32-word allocation unit. One allocation unit is exactly one ConcurrentMap slab.

GPU pool allocation

SlabAlloc/src/slab_alloc.cuh

Function/class: SlabAllocLight constructor

What it is doing: cudaMallocs one device pool: bitmap words are zeroed and allocation-unit content is set to all ones.

```
SlabAllocLight() : d_super_blocks_(nullptr), hash_coef_(0) {
    // random coefficients for allocator's hash function
    std::mt19937 rng(time(0));
    hash_coef_ = rng();

    // In the light version, we put num_super_blocks super blocks within a
    // single array
    CHECK_ERROR(cudaMalloc((void**)&d_super_blocks_,
                           slab_alloc_context_.SUPER_BLOCK_SIZE_ *
                           slab_alloc_context_.num_super_blocks_ * sizeof(uint32_t)));

    for (int i = 0; i < slab_alloc_context_.num_super_blocks_; i++) {
        // setting bitmaps into zeros:
        CHECK_ERROR(cudaMemset(d_super_blocks_ + i * slab_alloc_context_.SUPER_BLOCK_SIZE_,
                               0x00,
                               slab_alloc_context_.NUM_MEM_BLOCKS_PER_SUPER_BLOCK_ *
                               slab_alloc_context_.BITMAP_SIZE_ * sizeof(uint32_t)));
        // setting empty memory units into ones:
        CHECK_ERROR(cudaMemset(d_super_blocks_ + i * slab_alloc_context_.SUPER_BLOCK_SIZE_ +
                               (slab_alloc_context_.NUM_MEM_BLOCKS_PER_SUPER_BLOCK_ *
                               slab_alloc_context_.BITMAP_SIZE_),
                               0xFF,
                               slab_alloc_context_.MEM_BLOCK_SIZE_ *
                               slab_alloc_context_.NUM_MEM_BLOCKS_PER_SUPER_BLOCK_ *
                               sizeof(uint32_t)));
    }

    // initializing the slab context:
    slab_alloc_context_.initParameters(d_super_blocks_, hash_coef_);
}

// =====
// destructor:
// =====
```

No insertion cudaMallocs memory. SlabAlloc reserves a GPU pool during host construction. A zero bitmap bit means free, and the all-one contents make new units immediately valid empty slabs.

Packed slab addresses

SlabAlloc/src/slab_alloc.cuh

Function/class: SlabAllocLightContext address helpers

What it is doing: Defines packed superblock/block/unit fields and address decoding.

```
SUPER_BLOCK_BIT_OFFSET_ALLOC_ = 27;
static constexpr uint32_t MEM_BLOCK_BIT_OFFSET_ALLOC_ = 10;
static constexpr uint32_t MEM_UNIT_BIT_OFFSET_ALLOC_ = 5;
static constexpr uint32_t NUM_MEM_BLOCKS_PER_SUPER_BLOCK_ = (1 << LOG_NUM_MEM_BLOCKS_);
static constexpr uint32_t MEM_BLOCK_SIZE_ = NUM_MEM_UNITS_PER_BLOCK_ * MEM_UNIT_SIZE_;
static constexpr uint32_t SUPER_BLOCK_SIZE_ =
    ((BITMAP_SIZE_ + MEM_BLOCK_SIZE_) * NUM_MEM_BLOCKS_PER_SUPER_BLOCK_);
static constexpr uint32_t MEM_BLOCK_OFFSET_ =
    (BITMAP_SIZE_ * NUM_MEM_BLOCKS_PER_SUPER_BLOCK_);
static constexpr uint32_t num_super_blocks_ = NUM_SUPER_BLOCKS_ALLOCATOR_;

__device__ __host__ SlabAllocLightContext()
: d_super_blocks_(nullptr)
, hash_coef_(0)
, num_attempts_(0)
, resident_index_(0)
, super_block_index_(0)
, allocated_index_(0) {}

__device__ __host__ SlabAllocLightContext(const SlabAllocLightContext& rhs)
: d_super_blocks_(rhs.d_super_blocks_)
, hash_coef_(rhs.hash_coef_)
, num_attempts_(0)
```

```

    , resident_index_(0)
    , super_block_index_(0)
    , allocated_index_(0) {}

SlabAllocLightContext& operator=(const SlabAllocLightContext& rhs) {
    d_super_blocks_ = rhs.d_super_blocks_;
    hash_coef_ = rhs.hash_coef_;
    num_attempts_ = 0;
    resident_index_ = 0;
    super_block_index_ = 0;
    allocated_index_ = 0;
    return *this;
}

__device__ __host__ ~SlabAllocLightContext() {}

__device__ __host__ void initParameters(uint32_t* d_super_block, uint32_t hash_coef) {
    d_super_blocks_ = d_super_block;
    hash_coef_ = hash_coef;
}

__device__ __host__ void copyParameters(const SlabAllocLightContext& rhs) {
    d_super_blocks_ = rhs.d_super_blocks_;
    hash_coef_ = rhs.hash_coef_;
}

// =====
// some helper inline address functions:
// =====
__device__ __host__ __forceinline__ uint32_t
getSuperBlockIndex(SlabAllocAddressT address) const {

```

The chain link is a compact uint32 address, not a raw pointer. `getPointerFromSlab` decodes its allocation unit base and adds `laneId`, so all 32 lanes address one 32-word slab.

How a warp obtains a slab

SlabAlloc/src/slab_alloc.cuh

Function/class: SlabAllocLightContext::warpAllocate

What it is doing: Elects a lane with a free bitmap bit. CAS-claims it. then broadcasts the packed address.

```

uint32_t warpAllocate(const uint32_t& laneId) {
    // tries and allocate a new memory units within the resident memory block
    // if it returns 0xFFFFFFFF, then there was not any empty memory unit
    // a new resident block should be chosen, and repeat again
    // allocated result:  5 bits: super_block_index
    //                  17 bits: memory block index
    //                  5 bits: memory unit index (hi-bits of 10bit)
    //                  5 bits: memory unit index (lo-bits of 10bit)
    int empty_lane = -1;
    uint32_t free_lane;
    uint32_t read_bitmap = resident_bitmap_;
    uint32_t allocated_result = 0xFFFFFFFF;
    // works as long as <31 bit are used in the allocated_result
    // in other words, if there are 32 super blocks and at most 64k blocks per
    // super block

    while (allocated_result == 0xFFFFFFFF) {
        empty_lane = __ffs(~resident_bitmap_) - 1;
        free_lane = __ballot_sync(0xFFFFFFFF, empty_lane >= 0);
        if (free_lane == 0) {
            // all bitmaps are full: need to be rehashed again:
            updateMemBlockIndex((threadIdx.x + blockIdx.x * blockDim.x) >> 5);
            read_bitmap = resident_bitmap_;
            continue;
        }
        uint32_t src_lane = __ffs(free_lane) - 1;
        if (src_lane == laneId) {
            read_bitmap = atomicCAS(d_super_blocks_ + super_block_index_ * SUPER_BLOCK_SIZE_ +
                                   resident_index_ * BITMAP_SIZE_ + laneId,
                                   resident_bitmap_,
                                   resident_bitmap_ | (1 << empty_lane));
            if (read_bitmap == resident_bitmap_) {
                // successful attempt:
                resident_bitmap_ |= (1 << empty_lane);
                allocated_result = (super_block_index_ << SUPER_BLOCK_BIT_OFFSET_ALLOC_) |
                                   (resident_index_ << MEM_BLOCK_BIT_OFFSET_ALLOC_) |
                                   (laneId << MEM_UNIT_BIT_OFFSET_ALLOC_) | empty_lane;
            } else {
                // Not successful: updating the current bitmap
                resident_bitmap_ = read_bitmap;
            }
        }
    }
    // asking for the allocated result;

```

```

    allocated_result = __shfl_sync(0xFFFFFFFF, allocated_result, src_lane);
}
return allocated_result;
}

__device__ __forceinline__ uint32_t warpAllocateBulk(uint32_t& laneId,
const uint32_t k) {
    // tries and allocate k consecutive memory units within the resident memory
    // block if it returns 0xFFFFFFFF, then there was not any empty memory unit
    // a new resident block should be chosen, and repeat again
    // allocated result: 5 bits: super_block_index
    //                  17 bits: memory block index
    //                  5 bits: memory unit index (hi-bits of 10bit)
    //                  5 bits: memory unit index (lo-bits of 10bit)
    int empty_lane = -1;
    uint32_t free_lane;
    uint32_t read_bitmap = resident_bitmap_;
    uint32_t allocated_result = 0xFFFFFFFF;
    // works as long as <31 bit are used in the allocated_result

```

initAllocator hashes a global warp ID to a resident memory block and loads one bitmap word per lane. bitmap atomicCAS prevents two requesters who observe one free bit from both claiming it. Failure reloads state; a full block makes updateMemBlockIndex choose another. The elected lane shuffles the one resulting address to every lane.

Recycling limitation

SlabAlloc/src/slab_alloc.cuh

Function/class: SlabAllocLightContext::freeUntouched

What it is doing: Clears a bitmap bit only for an allocated but never-published candidate.

```

void freeUntouched(SlabAllocAddressT ptr) {
    atomicAnd(d_super_blocks_ + getSuperBlockIndex(ptr) * SUPER_BLOCK_SIZE_ +
        getMemBlockIndex(ptr) * BITMAP_SIZE_ + (getMemUnitIndex(ptr) >> 5),
        ~(1 << (getMemUnitIndex(ptr) & 0x1F)));
}

__host__ __device__ __forceinline__ SlabAllocAddressT

```

insertPair invokes freeUntouched after losing a link CAS. deleteKey does not unlink/free used overflow slabs; it merely empties a pair. Chains do not shrink.

6. ConcurrentMap operations

Host API launches actual CUDA kernels

src/concurrent_map/cmap_implementation.cuh

Function/class: buildBulk / searchIndividual / deleteIndividual

What it is doing: Computes 128-thread grids and launches kernels with `gpu_context_`.

```
void GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::buildBulk(
    KeyT* d_key,
    ValueT* d_value,
    uint32_t num_keys) {
    const uint32_t num_blocks = (num_keys + BLOCKSIZE_ - 1) / BLOCKSIZE_;
    // calling the kernel for bulk build:
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));
    build_table_kernel<KeyT, ValueT>
        <<<num_blocks, BLOCKSIZE_>>>(d_key, d_value, num_keys, gpu_context_);
}

template <typename KeyT, typename ValueT>
void GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::buildBulkWithUniqueKeys(
    KeyT* d_key,
    ValueT* d_value,
    uint32_t num_keys) {
    const uint32_t num_blocks = (num_keys + BLOCKSIZE_ - 1) / BLOCKSIZE_;
    // calling the kernel for bulk build:
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));
    build_table_with_unique_keys_kernel<KeyT, ValueT>
        <<<num_blocks, BLOCKSIZE_>>>(d_key, d_value, num_keys, gpu_context_);
}

template <typename KeyT, typename ValueT>
void GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::searchIndividual(
    KeyT* d_query,
    ValueT* d_result,
    uint32_t num_queries) {
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));
    const uint32_t num_blocks = (num_queries + BLOCKSIZE_ - 1) / BLOCKSIZE_;
    search_table<KeyT, ValueT>
        <<<num_blocks, BLOCKSIZE_>>>(d_query, d_result, num_queries, gpu_context_);
}

template <typename KeyT, typename ValueT>
void GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::searchBulk(
    KeyT* d_query,
    ValueT* d_result,
    uint32_t num_queries) {
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));
    const uint32_t num_blocks = (num_queries + BLOCKSIZE_ - 1) / BLOCKSIZE_;
    search_table_bulk<KeyT, ValueT>
        <<<num_blocks, BLOCKSIZE_>>>(d_query, d_result, num_queries, gpu_context_);
}

template <typename KeyT, typename ValueT>
void GpuSlabHash<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::countIndividual(
    KeyT* d_query,
    uint32_t* d_count,
    uint32_t num_queries) {
    CHECK_CUDA_ERROR(cudaSetDevice(device_idx_));
    const uint32_t num_blocks = (num_queries + BLOCKSIZE_ - 1) / BLOCKSIZE_;
    count_key<KeyT, ValueT>
```

`BLOCKSIZE_` is 128 = four warps. `computeBucket` implements $(((\text{hash_x_} \wedge \text{key}) + \text{hash_y_}) \% 4294967291) \% \text{num_buckets_}$. Coefficients are randomized unless `identity_hash` is true.

Insert call path

Step	Code flow
1	buildBulk -> build_table_kernel: tid reads d_key/d_value, computes bucket, marks to_insert.
2	Kernel creates a local allocator context, initializes it, calls context.insertPair.
3	insertPair ballots pending requests, chooses src_lane; all lanes load one source slab word.
4	Free pair: source lane 64-bit pair CAS. Full terminal slab: warpAllocate then lane-31 link CAS.
5	Existing link: follow encoded slab and repeat. Completed request clears its flag.

src/concurrent_map/warp/insert.cuh

Function/class: GpuSlabHashContext::insertPair

What it is doing: Core empty-slot ballot. whole-pair CAS and lane-31 overflow-link publication.

```
uint32_t isEmpty = (__ballot_sync(0xFFFFFFFF, src_unit_data == EMPTY_KEY)) &
    SlabHashT::REGULAR_NODE_KEY_MASK;
if (isEmpty == 0) { // no empty slot available:
    uint32_t next_ptr = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);
    if (next_ptr == SlabHashT::EMPTY_INDEX_POINTER) {
        // allocate a new node:
        uint32_t new_node_ptr = allocateSlab(local_allocator_ctx, laneId);

        // TODO: experiment if it's better to use lane 0 instead
        if (laneId == 31) {
            const uint32_t* p = (next == SlabHashT::A_INDEX_POINTER)
                ? getPointerFromBucket(src_bucket, 31)
                : getPointerFromSlab(next, 31);

            uint32_t temp =
                atomicCAS((unsigned int*)p, SlabHashT::EMPTY_INDEX_POINTER, new_node_ptr);
            // check whether it was successful, and
            // free the allocated memory otherwise
            if (temp != SlabHashT::EMPTY_INDEX_POINTER) {
                freeSlab(new_node_ptr);
            }
        }
        next = next_ptr;
    }
} else { // there is an empty slot available
    int dest_lane = __ffs(isEmpty & SlabHashT::REGULAR_NODE_KEY_MASK) - 1;
    if (laneId == src_lane) {
        const uint32_t* p = (next == SlabHashT::A_INDEX_POINTER)
            ? getPointerFromBucket(src_bucket, dest_lane)
            : getPointerFromSlab(next, dest_lane);

        old_key_value_pair =
            atomicCAS((unsigned long long int*)p,
                EMPTY_PAIR_64,
                ((uint64_t)(*reinterpret_cast<const uint32_t*>(&myValue))
                    reinterpret_cast<const unsigned char*>(&myValue)))
                    << 32) |
                *reinterpret_cast<const uint32_t*>(&myKey));
        if (old_key_value_pair == EMPTY_PAIR_64)
            to_be_inserted = false; // successful insertion
    }
}
```

64-bit CAS prevents a torn key/value entry. Plain insertPair permits duplicate keys. insertPairUnique adds an existing-key ballot and suppresses a duplicate. Link CAS makes one overflow node publication win; a loser returns its unpublished candidate.

6. ConcurrentMap: search, delete, count

Search path

src/concurrent_map/warp/search.cuh

Function/class: GpuSlabHashContext::searchKey

What it is doing: Ballots matching key lanes and either shuffles following value word or follows lane-31 link.

```
const uint32_t src_unit_data = (next == SlabHashT::A_INDEX_POINTER)
                                ? *(getPointerFromBucket(src_bucket, laneId))
                                : *(getPointerFromSlab(next, laneId));

int found_lane = __ffs(__ballot_sync(0xFFFFFFFF, src_unit_data == wanted_key) &
                      SlabHashT::REGULAR_NODE_KEY_MASK) -
1;

if (found_lane < 0) { // not found
    uint32_t next_ptr = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);
    if (next_ptr == SlabHashT::EMPTY_INDEX_POINTER) { // not found
        if (laneId == src_lane) {
            myValue = static_cast<ValueT>(SEARCH_NOT_FOUND);
            to_be_searched = false;
        }
    } else {
        next = next_ptr;
    }
} else { // found the key:
    uint32_t found_value = __shfl_sync(0xFFFFFFFF, src_unit_data, found_lane + 1, 32);
    if (laneId == src_lane) {
        myValue = *reinterpret_cast<const ValueT*>(
            reinterpret_cast<const unsigned char*>(&found_value));
        to_be_searched = false;
    }
}
last_work_queue = work_queue;
}
}

//=====
// Bulk Search Unit:
//=====
template <typename KeyT, typename ValueT>
__device__ __forceinline__ void
GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap>::searchKeyBulk(
    const uint32_t& laneId,
```

A search only misses when lane 31 supplies EMPTY_INDEX_POINTER. found_lane is an even key lane; found_lane+1 is its value. Plain map insertion permits duplicates, so search returns the first match in base-to-overflow / lowest-key-lane order. searchKeyBulk uses the same storage but loops source lanes 0..31 explicitly.

Delete path

src/concurrent_map/warp/delete.cuh

Function/class: GpuSlabHashContext::deleteKey

What it is doing: Finds a key with the same chain traversal then atomically replaces the whole pair with EMPTY_PAIR_64.

```
uint32_t isFound = (__ballot_sync(0xFFFFFFFF, src_unit_data == src_key)) &
                  SlabHashT::REGULAR_NODE_KEY_MASK;

if (isFound == 0) { // no matching slot found:
    uint32_t next_ptr = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);
    if (next_ptr == SlabHashT::EMPTY_INDEX_POINTER) {
        // not found:
        if (laneId == src_lane)
            to_be_deleted = false;
    } else {
        next = next_ptr;
    }
} else { // The wanted key found:
    int dest_lane = __ffs(isFound & SlabHashT::REGULAR_NODE_KEY_MASK) - 1;
    if (laneId == src_lane) {
        uint32_t* p = (next == SlabHashT::A_INDEX_POINTER)
            ? getPointerFromBucket(src_bucket, dest_lane)
            : getPointerFromSlab(next, dest_lane);

        uint64_t old_pair = atomicExch((unsigned long long int*)p, EMPTY_PAIR_64);
        uint32_t deleted_key = static_cast<uint32_t>(old_pair);
        successful_deletion = deleted_key == src_key;
        to_be_deleted = false;
    }
}
}
```

```
    last_work_queue = work_queue;
}
return successful_deletion;
}
```

atomicExch avoids torn removal. The returned low word is checked because a concurrent mutation may follow the earlier ballot. No link is modified and no slab is freed. countKey uses this same traversal and __popc on a key-match ballot, so it counts duplicates.

7. Warp-level execution and concurrency

Phase	All lanes do	Source lane does
Queue	__ballot_sync makes pending mask; __ffs elects one lane.	Its request is selected.
Load	Read word laneId of selected request's slab.	Bucket/key broadcast by shuffle.
Decide	Ballot comparisons make match/free masks.	Receives value/completion.
Mutate	Participate in allocator; lane 31 publishes link.	Pair CAS/exchange.

src/concurrent_map/device/build.cuh

Function/class: build_table_kernel

What it is doing: Computes tid/laneId and preserves partial final warp participation for full-mask intrinsics.

```
uint32_t tid = threadIdx.x + blockIdx.x * blockDim.x;
uint32_t laneId = threadIdx.x & 0x1F;

if ((tid - laneId) >= num_keys) {
    return;
}

AllocatorContextT local_allocator_ctx(slab_hash.getAllocatorContext());
local_allocator_ctx.initAllocator(tid, laneId);

KeyT myKey = 0;
ValueT myValue = 0;
uint32_t myBucket = 0;
bool to_insert = false;

if (tid < num_keys) {
    myKey = d_key[tid];
    myValue = d_value[tid];
    myBucket = slab_hash.computeBucket(myKey);
    to_insert = true;
}
```

The guard checks warp base, not just tid: valid tail requests need invalid sibling lanes for full-mask ballot/shuffle participation. A warp may hold keys for many buckets; it handles one selected source request at a time, then returns to its work queue.

Mechanism	Code site	Race prevented
64-bit atomicCAS	insert pair	Two inserts select one pair; partial pair publication.
32-bit atomicCAS	lane-31 link	Two overflowing inserts attach different next slabs.
bitmap atomicCAS	warpAllocate	Two allocations claim one unit.
atomicAnd	freeUntouched	Only loser candidate bit cleared.
atomicExch	delete pair	Partial pair delete.

This source relies on CUDA intrinsic atomics and warp intrinsics; it contains no C++ memory-order arguments or explicit fences in these paths.

8. GpuSlabHashContext

Context exists because a kernel needs a copyable device handle, not an owner with cudaMalloc/free lifetime. It is copied by value into kernels and owns no GPU allocation.

src/concurrent_map/cmap_class.cuh

Function/class: GpuSlabHashContext::initParameters / computeBucket

What it is doing: Stores map metadata. d_table_ reference and copied allocator context: maps keys to buckets.

```
void initParameters(const uint32_t num_buckets,
                   const uint32_t hash_x,
                   const uint32_t hash_y,
                   int8_t* d_table,
                   AllocatorContextT* allocator_ctx) {
    num_buckets_ = num_buckets;
    hash_x_ = hash_x;
    hash_y_ = hash_y;
    d_table_ =
        reinterpret_cast<typename ConcurrentMapT<KeyT, ValueT>::SlabTypeT*>(d_table);
    global_allocator_ctx_ = *allocator_ctx;
}

__device__ __host__ __forceinline__ AllocatorContextT& getAllocatorContext() {
    return global_allocator_ctx_;
}

__device__ __host__ __forceinline__ typename ConcurrentMapT<KeyT, ValueT>::SlabTypeT*
getDeviceTablePointer() {
    return d_table_;
}

__device__ __host__ __forceinline__ uint32_t getNumBuckets() { return num_buckets_; }
__device__ __host__ __forceinline__ uint32_t getHashX() { return hash_x_; }
__device__ __host__ __forceinline__ uint32_t getHashY() { return hash_y_; }

__device__ __host__ __forceinline__ uint32_t computeBucket(const KeyT& key) const {
    return (((hash_x_ ^ key) + hash_y_) % PRIME_DIVISOR_) % num_buckets_;
}

// threads in a warp cooperate with each other to insert key-value pairs
// into the slab hash
__device__ __forceinline__ void insertPair(bool& to_be_inserted,
                                           const uint32_t& laneId,
                                           const KeyT& myKey,
                                           const ValueT& myValue,
```

Its members are num_buckets_, hash_x_, hash_y_, typed d_table_, and global_allocator_ctx_. Scalars are copied. d_table_ refers to map-owned GPU storage; allocator context points into allocator-owned d_super_blocks_. The context exposes pointer access, computeBucket and device insert/search/delete/count; private allocateSlab/freeSlab connect to SlabAlloc.

GpuSlabHash construction calls initParameters with dynamic_allocator_->getContextPtr(). Host methods launch kernels with gpu_context_; kernels call context methods directly.

9. Variants

Variant	Actual difference	Coverage
ConcurrentMap	15 key/value pairs + two pointer words; plain/unique insert, search, bulk search, delete, count, batches.	Complete main variant.
ConcurrentSet	31 keys + one link; existing key suppresses insertion; 32-bit key CAS.	Build/individual search; bulk search/delete bodies empty.
PhaseConcurrentMap	Declared phase_concurrent_slab combines key-only slab and values.	No specialization source found.

src/slab_hash_global.cuh

Function/class: key_only_slab

What it is doing: Defines the ConcurrentSet's 31 data-key plus one link layout.

```
struct __align__(32) key_only_slab {
    static constexpr uint32_t NUM_ELEMENTS_PER_SLAB = 31u;
    KeyT keys[NUM_ELEMENTS_PER_SLAB];
    uint32_t next_ptr_index[1];
};
```

src/concurrent_set/cset_warp_operations.cuh

Function/class: GpuSlabHashContext::insertKey

What it is doing: Finds an existing key or empty slot, making set insertion idempotent.

```
int32_t dest_lane = SlabHash_NS::findKeyOrEmptyPerWarp<KeyT, ConcurrentSetT<KeyT>>(
    src_key, src_unit_data);

if (dest_lane == -1) { // key not found and/or no empty slot available:
    uint32_t next_ptr = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);
    if (next_ptr == SlabHashT::EMPTY_INDEX_POINTER) {
        // allocate a new node:
        uint32_t new_node_ptr = allocateSlab(local_allocator_ctx, laneId);

        if (laneId == 31) {
            uint32_t* p = (next == SlabHashT::A_INDEX_POINTER)
                ? getPointerFromBucket(src_bucket, 31)
                : getPointerFromSlab(next, 31);

            uint32_t temp =
                atomicCAS((unsigned int*)p, SlabHashT::EMPTY_INDEX_POINTER, new_node_ptr);
            // check whether it was successful, and
            // free the allocated memory otherwise
            if (temp != SlabHashT::EMPTY_INDEX_POINTER)
                freeSlab(new_node_ptr);
        }
    } else {
        next = next_ptr;
    }
```

The variants share allocator, hash calculation, base table, context/owner split and lane-31 chaining. Map's mask selects 15 pair-key lanes; set's 0x7FFFFFFF mask selects 31 keys. PhaseConcurrentMap behavior cannot be inferred from this checkout's declarations alone.

10. CUDA kernel walkthrough: individual search

This exact kernel shows the path from a user query through context to buckets/slabs and a result.

src/concurrent_map/device/search_kernel.cuh

Function/class: search_table

What it is doing: One thread initializes one query/result. while its warp cooperatively resolves queued queries.

```
__global__ void search_table(
    KeyT* d_queries,
    ValueT* d_results,
    uint32_t num_queries,
    GpuSlabHashContext<KeyT, ValueT, SlabHashTypeT::ConcurrentMap> slab_hash) {
    uint32_t tid = threadIdx.x + blockIdx.x * blockDim.x;
    uint32_t laneId = threadIdx.x & 0x1F;

    if ((tid - laneId) >= num_queries) {
        return;
    }

    KeyT myQuery = 0;
    ValueT myResult = static_cast<ValueT>(SEARCH_NOT_FOUND);
    uint32_t myBucket = 0;
    bool to_search = false;
    if (tid < num_queries) {
        myQuery = d_queries[tid];
        myBucket = slab_hash.computeBucket(myQuery);
        to_search = true;
    }

    slab_hash.searchKey(to_search, laneId, myQuery, myResult, myBucket);

    // writing back the results:
    if (tid < num_queries) {
        d_results[tid] = myResult;
    }
}

//=== Bulk search kernel:
template <typename KeyT, typename ValueT>
__global__ void search_table_bulk(
    KeyT* d_queries,
    ValueT* d_results,
```

tid selects d_queries[tid] and d_results[tid]. laneId is the low five bits of threadIdx. The warp-base guard retains full warp participation. Valid threads compute their own bucket and set to_search. searchKey selects a source lane with ballot, broadcasts that source key/bucket, scans a slab word-per-lane, follows links if needed, then completes the source request. Finally each valid thread writes its own result.

Layer	Exact crossing
Wrapper	gpu_hash_table::hash_search: host query -> d_query_; calls searchIndividual; copies d_results home.
Host map	GpuSlabHash::searchIndividual launches search_table(..., gpu_context_).
Kernel	search_table: d_queries[tid] -> computeBucket -> context.searchKey.
Context	searchKey: getPointerFromBucket, then getPointerFromSlab for each link.
GPU storage	d_table_base slab -> optional d_super_blocks_overflow slabs -> d_results[tid].

11. Memory layout

Allocation	Owner	Actual layout	
d_table_	GpuSlabHash	num_buckets contiguous 128-byte base slabs; bucket b begins at base+b*32 words.	
d_super_blocks_	SlabAllocLight	Bitmap metadata then 32-word units; links store packed addresses.	
d_key_/value/query/result/count	gpu_hash_table	Staging I/O, not entry storage.	
Node	words 0..29	word 30	word 31
d_table_ bucket b	15 key/value pairs	ptr_index[0]	empty or encoded A
d_super_blocks_ A	15 key/value pairs	ptr_index[0]	empty or encoded B
d_super_blocks_ B	same	same	...

src/concurrent_map/device/misc_kernels.cuh

Function/class: bucket_count_kernel

What it is doing: A warp counts base key words then walks the same lane-31 chain.

```
uint32_t pairs_count = 0;
uint32_t slabs_count = 1;

uint32_t src_unit_data = *slab_hash.getPointerFromBucket(wid, laneId);

pairs_count += __popc(__ballot_sync(0xFFFFFFFF, src_unit_data != EMPTY_KEY) &
    SlabHashT::REGULAR_NODE_KEY_MASK);
uint32_t next = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);

while (next != SlabHashT::EMPTY_INDEX_POINTER) {
    // counting pairs
    src_unit_data = *slab_hash.getPointerFromSlab(next, laneId);
    pairs_count += __popc(__ballot_sync(0xFFFFFFFF, src_unit_data != EMPTY_KEY) &
        SlabHashT::REGULAR_NODE_KEY_MASK);
    next = __shfl_sync(0xFFFFFFFF, src_unit_data, 31, 32);
    // counting slabs
    slabs_count++;
}
// writing back the results:
if (laneId == 0) {
    d_pairs_count_result[wid] = pairs_count;
    d_slabs_count_result[wid] = slabs_count;
}
}
```

This confirms a bucket is a physical first slab: count starts slabs_count=1 and directly addresses the bucket. It then follows only overflow addresses.

12. Benchmarks, tests and build

bench/main_benchmarks.cu parses -mode and dispatches to experiments.cuh. Scenarios use gpu_hash_table<uint32_t,uint32_t,ConcurrentMap>. The wrapper measures host API calls with CUDA events.

Mode	What is measured
0	singleton: fixed size/expected chain; build plus individual/bulk search.
1	fixed keys, changing expected chain/bucket count.
2	fixed expected chain, variable table/key size.
3	initial pure insert batches then mixed batches through batchedOperation.

bench/batched_data_gen.cpp

Function/class: BatchedDataGen::next_batch

What it is doing: Packs operation code in top two bits: search=3. insert=1. delete=2.

```
// search queries that actually exist in the data structure
// choosing the first num_search_exist_ from the beginning of the references:
// code 3 for search queries
for (int i = 0; i < num_search_exist_; i++) {
    h_batch_buffer_[output_offset + i] =
        (0xC0000000 | h_key_ref_[h_index_ref_[i]]);
}
output_offset += num_search_exist_;

// search queries that do not exist in the data structure
// choose the last num_search_non_exist_ from the end of the references:
// code 3 for search queries
for (int i = 0; i < num_search_non_exist_; i++) {
    h_batch_buffer_[output_offset + i] =
        (0xC0000000 | h_key_ref_[h_index_ref_[num_ref_ - i - 1]]);
}
output_offset += num_search_non_exist_;

// inserting new items:
// code 1:
// the first num_insert_ elements after the edge:
for (int i = 0; i < num_insert_; i++) {
    temp_buffer_[i] = h_index_ref_[edge_index_ + i];
    h_batch_buffer_[output_offset + i] =
        (0x40000000 | h_key_ref_[temp_buffer_[i]]);
}
output_offset += num_insert_;

// deleting previously inserted elements:
// code 2:
for (int i = 0; i < num_delete_; i++) {
    temp_buffer_[num_insert_ + i] = h_index_ref_[edge_index_ - i - 1];
    h_batch_buffer_[output_offset + i] =
        (0x80000000 | h_key_ref_[temp_buffer_[num_insert_ + i]]);
}

// shuffling the output buffer:
std::random_shuffle(h_batch_buffer_, h_batch_buffer_ + batch_size_);

// updating the edge index:
std::copy(temp_buffer_, temp_buffer_ + batch_size_,
          h_index_ref_ + edge_index_ - num_delete_);
edge_index_ += (num_insert_ - num_delete_);

batch_counter++;
return h_batch_buffer_;
}

void BatchedDataGen::print_batch() {
    printf("Batch %d:\n", batch_counter);
```

src/concurrent_map/device/concurrent_kernel.cuh

Function/class: batched_operations

What it is doing: Decodes operation code then executes warp-wide insert, delete and search phases.

```
myOperation = d_operations[tid];
myKey = myOperation & 0x3FFFFFFF;
myBucket = slab_hash.computeBucket(myKey);
```

```

    myOperation = myOperation >> 30;
    // todo: should be changed to a more general case
    myValue = myKey; // for the sake of this benchmark
}

bool to_insert = (myOperation == 1) ? true : false;
bool to_delete = (myOperation == 2) ? true : false;
bool to_search = (myOperation == 3) ? true : false;

// first insertions:
slab_hash.insertPair(to_insert, laneId, myKey, myValue, myBucket, local_allocator_ctx);

// second deletions:
slab_hash.deleteKey(to_delete, laneId, myKey, myBucket);

// finally search queries:
slab_hash.searchKey(to_search, laneId, myKey, myValue, myBucket);

if (myOperation == 3 && myValue != SEARCH_NOT_FOUND) {
    d_results[tid] = myValue;
}
}

```

A mixed benchmark batch is ordered: all requested inserts, then deletes, then searches. It is not arbitrary per-thread operation interleaving.

Test	Input / result	Code covered
BulkBuild.IndividualSearch	137 keys, 2 buckets, shuffled queries versus unordered_map.	Overflow chains/build/individual search.
BulkBuild.BulkSearch	Same data, bulk search validated.	searchKeyBulk.
BulkBuild.IndividualCount	2,014 unique keys with random multiplicities.	Duplicate insert/countKey.
UniqueBulkBuild.Individual Count	Same generator, expected 0/1.	insertPairUnique.
BulkBuild.IndividualDelete	137/2 then shuffled deletes, compared after erase.	deleteKey.

Build system

CMakeLists.txt

Function/class: FindCUDA/gencode setup

What it is doing: Requires CUDA 8.0. C++11 and optional gencode architecture choices.

```

cmake_minimum_required (VERSION 3.8 FATAL_ERROR)
project (SlabHash)

find_package(CUDA 8.0 REQUIRED)

option(CMAKE_VERBOSE_MAKEFILE ON)
option(DGTEST, "DGTEST" ON)

set(CUDA_NVCC_FLAGS -std=c++11)
set (CMAKE_CXX_STANDARD 11)

if (CUDA_VERBOSE_PTXAS)
    set(VERBOSE_PTXAS --ptxas-options=-v)
endif (CUDA_VERBOSE_PTXAS)

set(CMAKE_BUILD_TYPE "Release")

set(CMAKE_RUNTIME_OUTPUT_DIRECTORY ${CMAKE_BINARY_DIR}/bin)

set(GENCODE_SM30
    -gencode=arch=compute_30,code=sm_30 -gencode=arch=compute_30,code=compute_30)
set(GENCODE_SM35
    -gencode=arch=compute_35,code=sm_35 -gencode=arch=compute_35,code=compute_35)
set(GENCODE_SM37
    -gencode=arch=compute_37,code=sm_37 -gencode=arch=compute_37,code=compute_37)
set(GENCODE_SM50
    -gencode=arch=compute_50,code=sm_50 -gencode=arch=compute_50,code=compute_50)
set(GENCODE_SM60
    -gencode=arch=compute_60,code=sm_60 -gencode=arch=compute_60,code=compute_60)

```

```
set(GENCODE_SM61
  -gencode=arch=compute_61,code=sm_61 -gencode=arch=compute_61,code=compute_61)
set(GENCODE_SM70
  -gencode=arch=compute_70,code=sm_70 -gencode=arch=compute_70,code=compute_70)
set(GENCODE_SM71
```

CMakeLists.txt

Function/class: includes/subdirectories

What it is doing: Connects SlabAlloc and ThirdParty, then test/bench targets.

```
include_directories(SlabAlloc/src)
include_directories(src src/concurrent)
include_directories(ThirdParty/rapidjson/include)
include_directories(ThirdParty/googletest/googletest)
include_directories(ThirdParty/googletest/googletest/include)
add_subdirectory(ThirdParty/googletest/googletest)
add_subdirectory(test)
add_subdirectory(bench)

if (DGTEST)
  enable_testing()
endif()
```

README's build order is submodule init/update, build directory, cmake and make. The source defaults to SM35, a CUDA-8-era target; a current toolkit/GPU may need an appropriate architecture setting. No CUDA run was used for this source guide.

13. Complete call/data flow

Search

Step	Actual arrow
1	gpu_hash_table::hash_search: host query -> d_query_; calls searchIndividual.
2	GpuSlabHash::searchIndividual -> search_table(..., gpu_context_).
3	search_table: tid/laneld -> d_queries[tid] -> computeBucket -> context.searchKey.
4	searchKey: ballot queue -> base getPointerFromBucket -> key comparison -> value word or lane-31 link.
5	getPointerFromSlab decodes link into allocator d_super_blocks_ overflow unit.
6	myResult -> d_results[tid] -> cudaMemcpy host result.

Insert

Step	Actual arrow
1	hash_build -> buildBulk -> build_table_kernel after input copy to d_key_/d_value_.
2	Kernel local allocator context -> initAllocator -> insertPair.
3	insertPair scans slab. Empty pair -> 64-bit atomicCAS.
4	Full terminal slab -> warpAllocate bitmap CAS -> lane-31 link CAS.
5	Link winner publishes address; candidate loser freeUntouched.

Delete

Step	Actual arrow
1	hash_delete -> deleteIndividual -> delete_table_keys.
2	Kernel derives key/bucket -> context.deleteKey.
3	Same chain lookup -> atomicExch entire pair to EMPTY_PAIR_64.
4	No unlink/free; pair slot reusable.

14. Reading order

Order	Read next	Why
1	README.md	Intended high-level API/context.
2	src/slab_hash_global.cuh	Sentinels/layouts/variants/aliases.
3	SlabAlloc/src/slab_alloc.cuh	Overflow allocation/address before links.
4	src/concurrent_map/cmap_class.cuh	Owner/context/construction/pointers/hash.
5	device/search_kernel.cuh then warp/search.cuh	Cleanest cooperative path.
6	device/build.cuh then warp/insert.cuh	Allocator and atomics.
7	delete/count/misc kernels	Deletion/duplicates/instrumentation.
8	cmap_implementation + gpu_hash_table	Host API connections.
9	concurrent_set/* + iterator	Compare specialized layout.
10	bench/* + cmap_test.cu	Workloads/tests after core understanding.
11	CMakeLists.txt	Build topology/architecture flags.

SlabHash - Complete Mental Model

gpu_hash_table can own staging arrays and create DynamicAllocatorT plus GpuSlabHash. GpuSlabHash owns d_table_, a fixed GPU array of base slabs, and initializes a non-owning context. That context is copied into CUDA kernels and references map-owned d_table_ and allocator-owned d_super_blocks_.

Each thread supplies a request, but one warp conducts an operation. A ballot selects a source request. Its key/bucket are shuffled to all lanes; every lane reads the matching word of a 32-word slab. Ballots identify free/matching key positions. For ConcurrentMap, words 0..29 are 15 pairs and word 31 is the next address.

Free insert atomically installs key/value as a 64-bit pair. Full collision slab triggers bitmap allocation, then lane 31 atomically publishes a packed next address. Search follows the same links to a value; delete atomically empties a pair but never reclaims its chain slab.

Memory aid: fixed base slabs + allocator overflow slabs + warp-wide slab scan + atomic pair/link/bitmap updates. Dynamic means collision-chain expansion, not table resize or normal delete-time slab reclamation.

Audited source: repository README/CMake/src/bench/test plus SlabAlloc/src/slab_alloc.cuh. ThirdParty was inspected only for role. This is source-code analysis, not a CUDA runtime performance report.